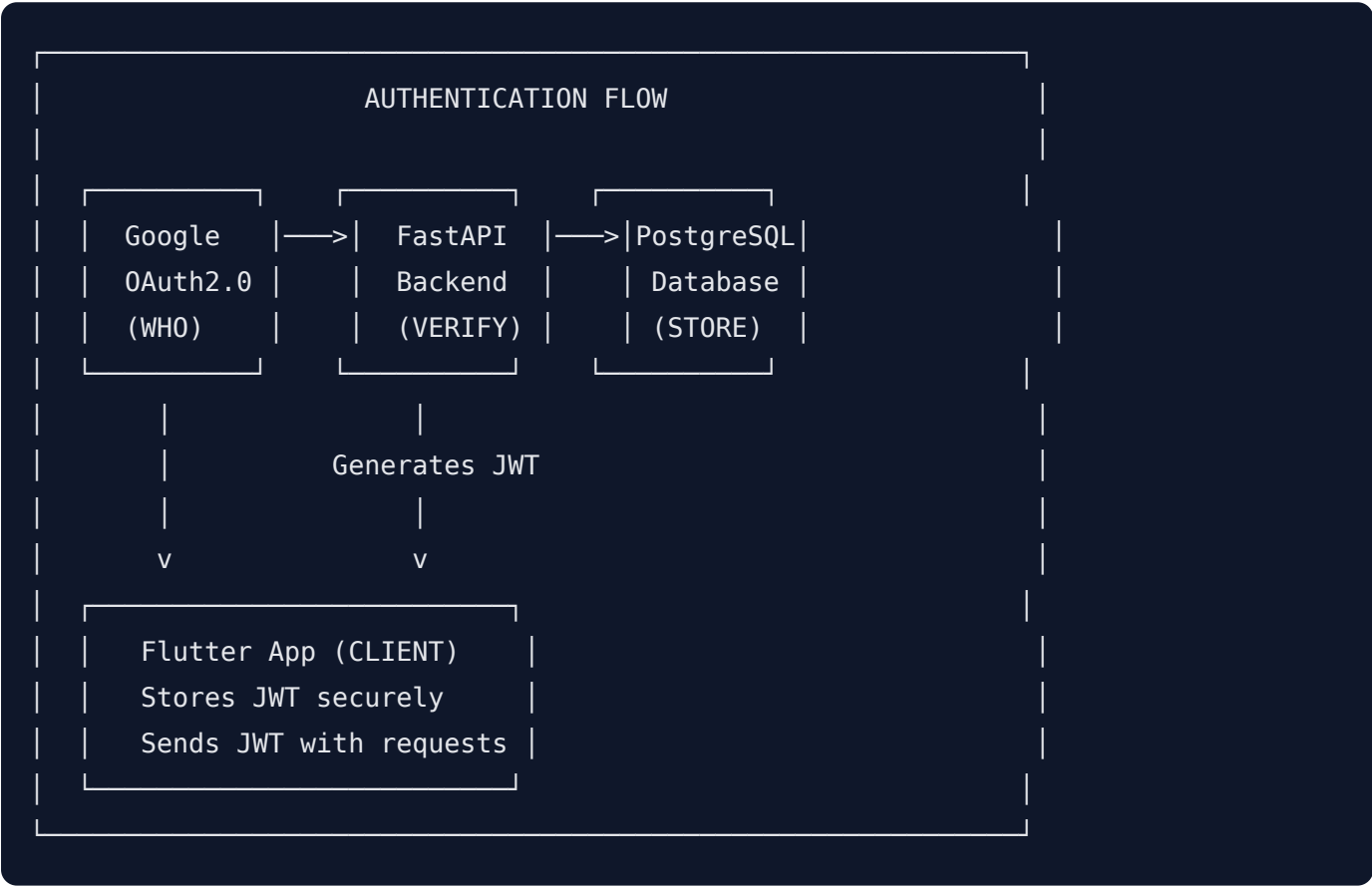


SyncSlash — Authentication & Authorization Deep-Dive

Complete Flow, Token Architecture, and Security Design

1. The Big Picture

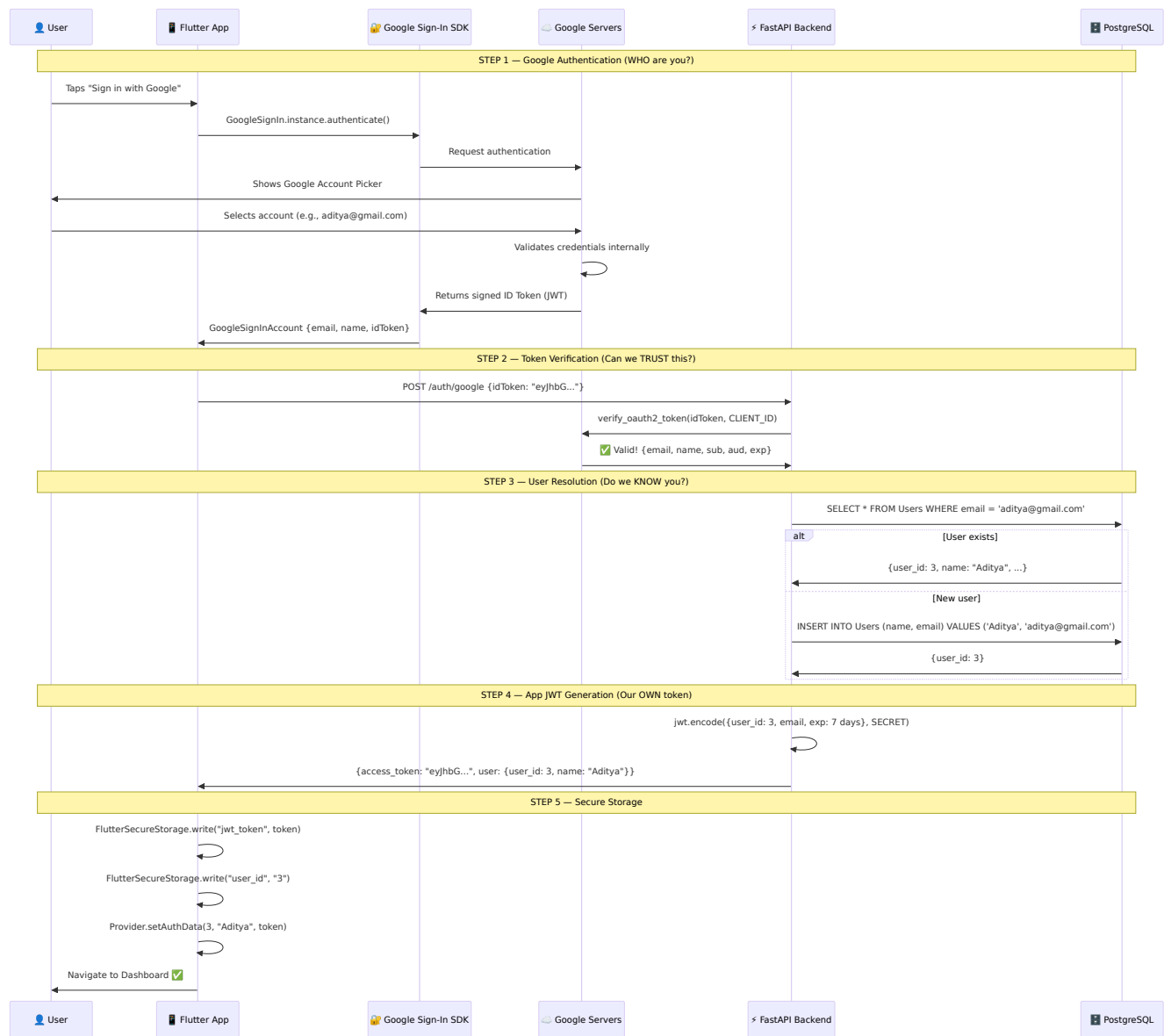
Our auth system has **three layers** working together:



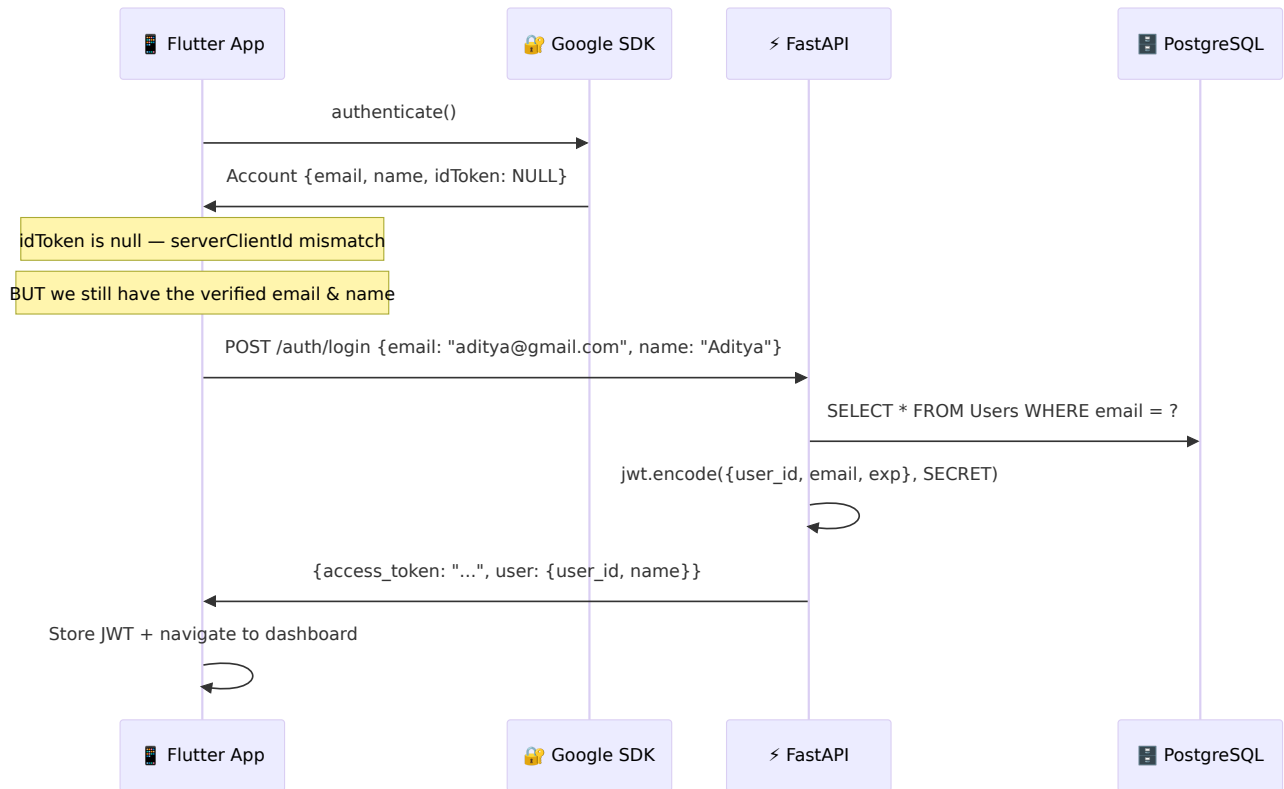
Layer	Role	Technology
Identity Provider	Proves WHO the user is	Google OAuth 2.0
Backend	Verifies identity, creates app user, issues JWT	FastAPI + PyJWT
Client	Stores token securely, sends with API calls	Flutter + FlutterSecureStorage
Database	Persists user records	PostgreSQL <code>Users</code> table

2. Complete Authentication Flow

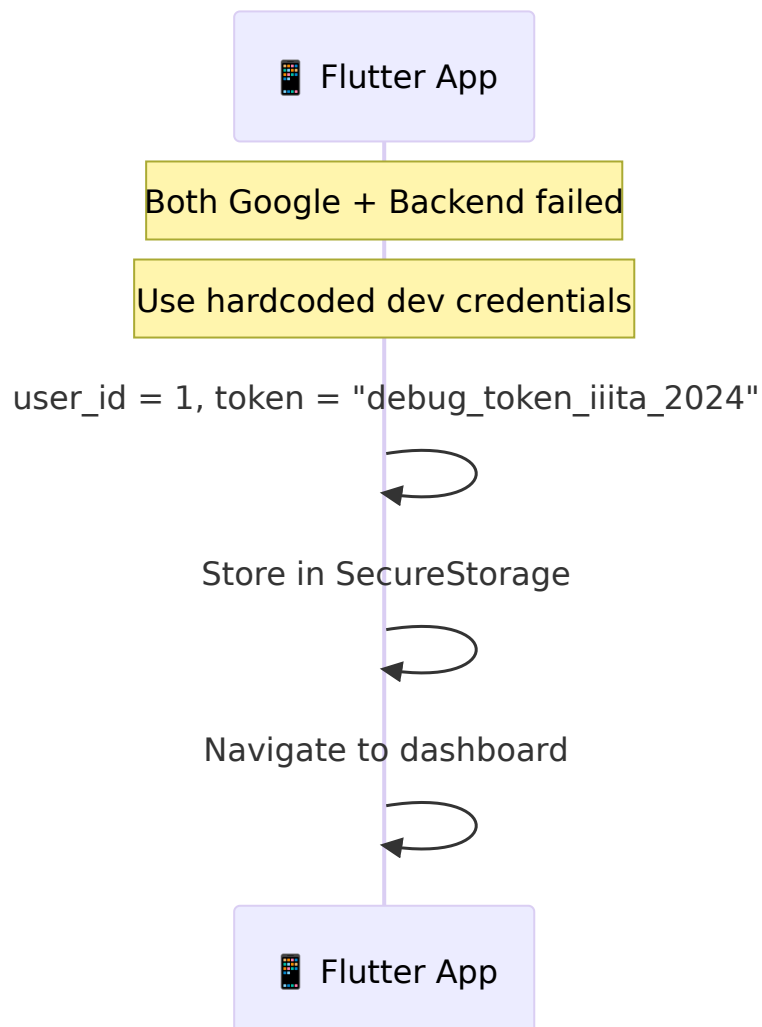
2.1 Primary Flow — Google OAuth + Backend Verification



2.2 Fallback Flow — Direct Login (When idToken is null)



2.3 Offline Fallback — Dev Mode (No backend needed)



3. Stateless vs Stateful — Our Design Choice

We use STATELESS authentication with JWT

STATEFUL (Sessions)

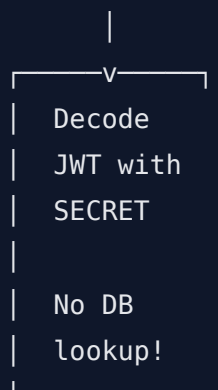
Client —cookie: session_id=abc123—> Server



- ✗ Server must store every active session
- ✗ Doesn't scale – need shared session store
- ✗ Not suited for mobile apps

STATELESS (JWT) ← WE USE THIS

Client —header: Bearer eyJhbG...—> Server



- ✓ Server stores NOTHING – token is self-contained
- ✓ Scales infinitely – any server can verify
- ✓ Perfect for mobile apps + REST APIs

Why Stateless for SyncSlash?

Factor	Stateful (Sessions)	Stateless (JWT) 
Server storage	Must store every session in Redis/DB	Nothing stored on server
Scalability	Need shared session store across servers	Any server can verify — just needs the SECRET key
Mobile apps	Cookies are awkward on mobile	Tokens sent in headers — natural for REST
Offline validation	Must hit server to validate	Token is self-contained — can decode locally
Performance	DB lookup on every request	Just cryptographic verification — $O(1)$
Render free tier	Would need Redis addon (\$\$\$)	Works with zero extra infrastructure

4. Token Deep-Dive

4.1 Google's ID Token (received from Google)

This is a JWT signed by Google's private key. We **don't generate** this — Google gives it to us.

```
eyJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJhY2NvdW50cy5nb29nbGUuY29tIiwiaW1h
aWwiOiJhZGl0eWFAZ21haWwuY29tIiwibmFtZSI6IkFkaXR5YSIsInN1YiI6IjExMj
M0NTY3ODkwIiwiaXVkiOiJoiNjUyMTc0NDQ5NzIyLXNpaHNnZHI4ZWZkb2I4aWRnM3A5
c2pkbjRlc2BzNXM4LmFwcHM4Z29vZ2xldXNlcmNvbnRlbnQuY29tIiwiaXhwIjoxNz
E1NjAwMDAwfQ.signature
```

Decoded payload:

```
{
  "iss": "accounts.google.com",           // Issuer: Google
  "email": "aditya@gmail.com",           // User's email
  "name": "Aditya Tomar",                // Display name
  "sub": "112345678901234567890",        // Google's unique user ID
  "aud": "652174449722-sihsgdr...com",   // OUR client ID (must match!)
  "iat": 1715596400,                      // Issued at
  "exp": 1715600000                        // Expires in ~1 hour
}
```

Verification on backend:

```
# backend/auth/routes.py
from google.oauth2 import id_token
from google.auth.transport import requests

idinfo = id_token.verify_oauth2_token(
    request.idToken,          # The token from Flutter
    requests.Request(),      # HTTP transport
    GOOGLE_CLIENT_ID         # Must match 'aud' in token
)
# If this doesn't throw → token is valid + not expired + meant for us
email = idinfo['email']      # Trusted – verified by Google
```

What `verify_oauth2_token` checks:

1. ☒ Signature valid (signed by Google's private key)
2. ☒ Not expired (`exp` > current time)
3. ☒ Audience matches our client ID (`aud` == our `CLIENT_ID`)
4. ☒ Issuer is Google (`iss` == "accounts.google.com")

4.2 Our App JWT (generated by backend)

After verifying with Google, we create our **own** JWT:

```
# backend/auth/routes.py
import jwt
import datetime

JWT_SECRET = os.getenv("JWT_SECRET", "tomar123") # Signing key

expiration = datetime.datetime.utcnow() + datetime.timedelta(days=7)

token = jwt.encode(
    {
        "user_id": 3,                # Our internal user ID
        "email": "aditya@gmail.com", # For reference
        "exp": expiration             # Expires in 7 days
    },
    JWT_SECRET,                     # Secret key for HMAC-SHA256
    algorithm="HS256"               # Symmetric signing
)
# token = "eyJhbGciOiJIUzI1NiIs..."
```

Why create our OWN token instead of using Google's?

Google's Token	Our Token
Expires in ~1 hour	Expires in 7 days
Contains Google-specific claims	Contains our <code>user_id</code>
Verification requires calling Google servers	Verification only needs our SECRET key
Can't customize payload	We control the payload

4.3 JWT Structure — Three Parts

Header.Payload.Signature

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9

← Header (base64)

.

eyJ1c2VyX2lkIjozLCJlbWFpbCI6ImFkaXR5...

← Payload (base64)

.

SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV...

← Signature (HMAC-SHA256)

Part	Content	Encoding
Header	<code>{"alg": "HS256", "typ": "JWT"}</code>	Base64URL (readable)
Payload	<code>{"user_id": 3, "email": "...", "exp": 1716201600}</code>	Base64URL (readable)
Signature	<code>HMAC-SHA256(header + "." + payload, SECRET)</code>	Binary (tamper-proof)

 **The payload is NOT encrypted** — it's just base64 encoded. Anyone can decode it. The **signature** prevents tampering. If someone changes `user_id: 3` to `user_id: 1`, the signature won't match and verification fails.

4.4 Signing Algorithm — HS256

HMAC-SHA256:

Input: base64(header) + "." + base64(payload)

Key: "tomar123" (our JWT_SECRET)

Output: 256-bit signature

To verify:

1. Take the header + payload from the token
2. Re-compute HMAC-SHA256 with the same SECRET
3. Compare with the signature in the token
4. If they match → token is authentic + untampered

HS256 = Symmetric → Same key to sign AND verify (simple, fast)

RS256 = Asymmetric → Private key signs, public key verifies (Google uses this)

5. Where Tokens Are Stored

5.1 Client Side — FlutterSecureStorage

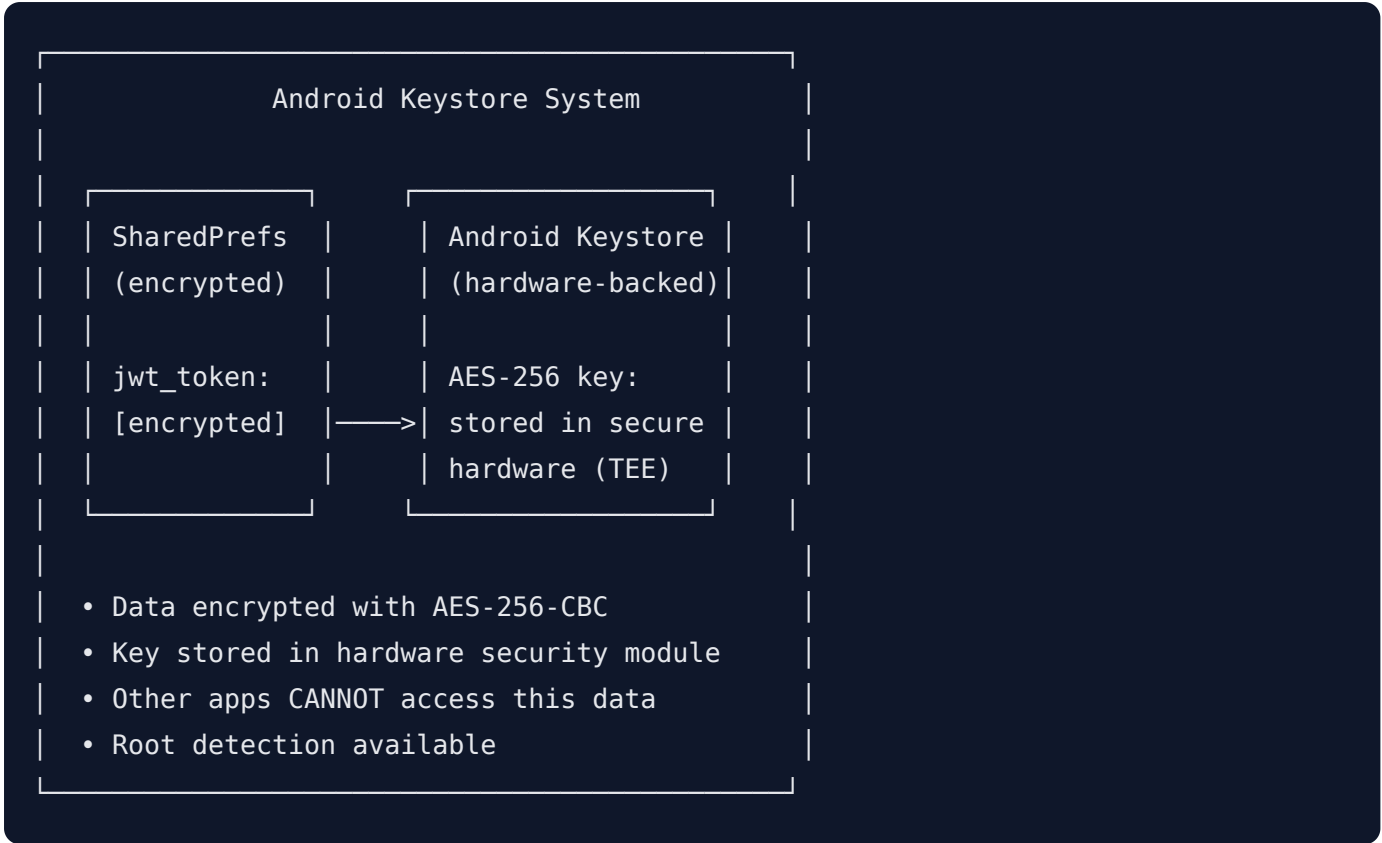
```
final storage = const FlutterSecureStorage();

// WRITE – after successful login
await storage.write(key: 'jwt_token', value: token);
await storage.write(key: 'user_id', value: user['user_id'].toString());

// READ – on app restart (check if already logged in)
String? savedToken = await storage.read(key: 'jwt_token');

// DELETE – on logout
await storage.deleteAll();
```

How FlutterSecureStorage works on Android:



Storage Method	Security Level	Used By
SharedPreferences	✗ Plain text, any root app can read	Bad practice
FlutterSecureStorage	✓ AES-256 encrypted, hardware-backed key	Our app
SQLite	✗ Plain text file on disk	Not for tokens

5.2 Server Side — NOTHING Stored

Server stores: NOTHING about active sessions

Why? Because JWT is SELF-CONTAINED:

- user_id is IN the token payload
- expiration is IN the token payload
- authenticity is proven by the SIGNATURE

The server only needs the SECRET KEY to verify any token.
No database lookup. No Redis. No session table.

5.3 In-Memory — Provider State

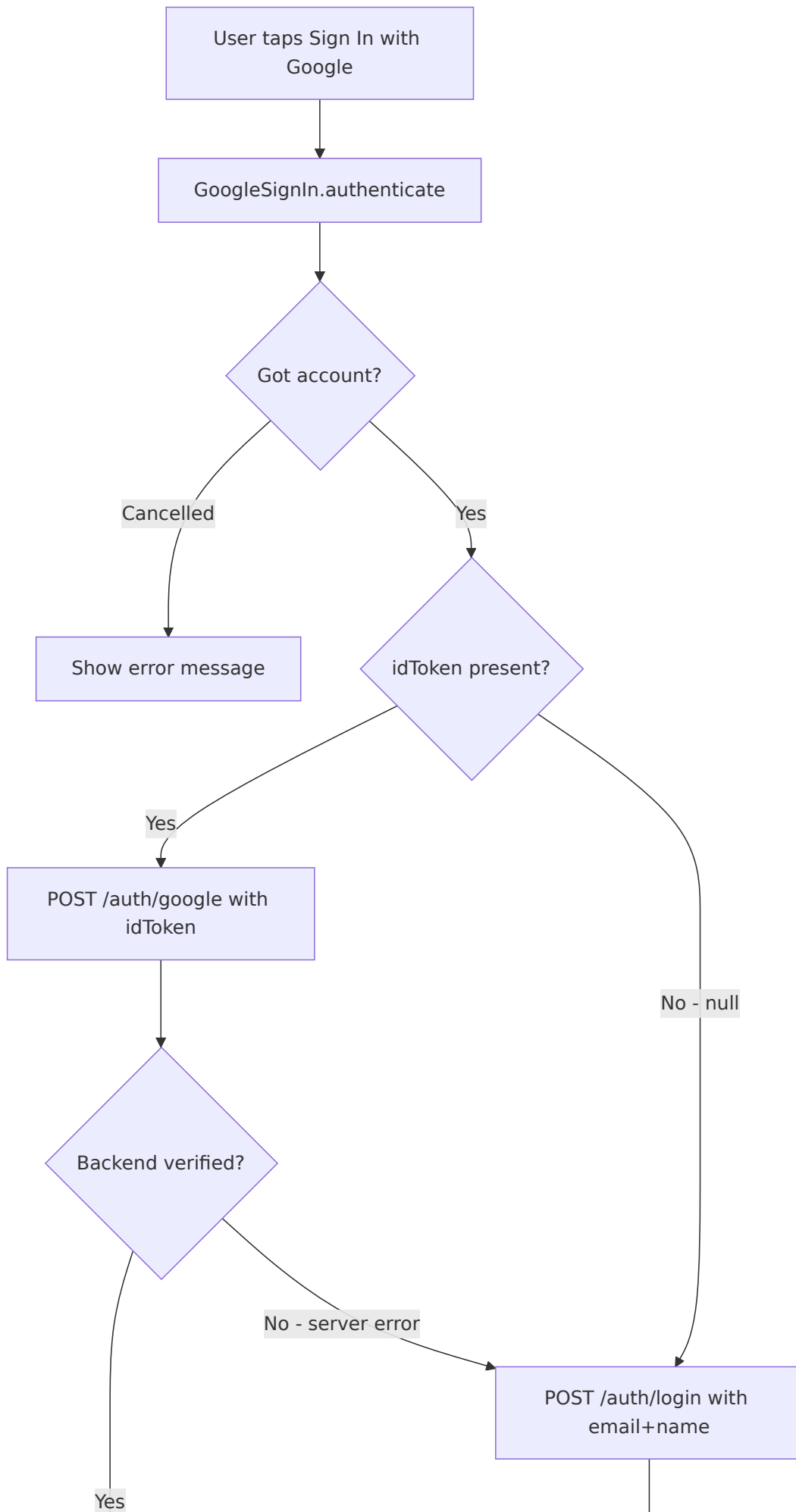
```
// state/app_provider.dart
class AppProvider with ChangeNotifier {
  int currentUserId = 1;      // From JWT payload
  String userName = 'User';   // From Google account
  String? jwtToken;           // The JWT itself

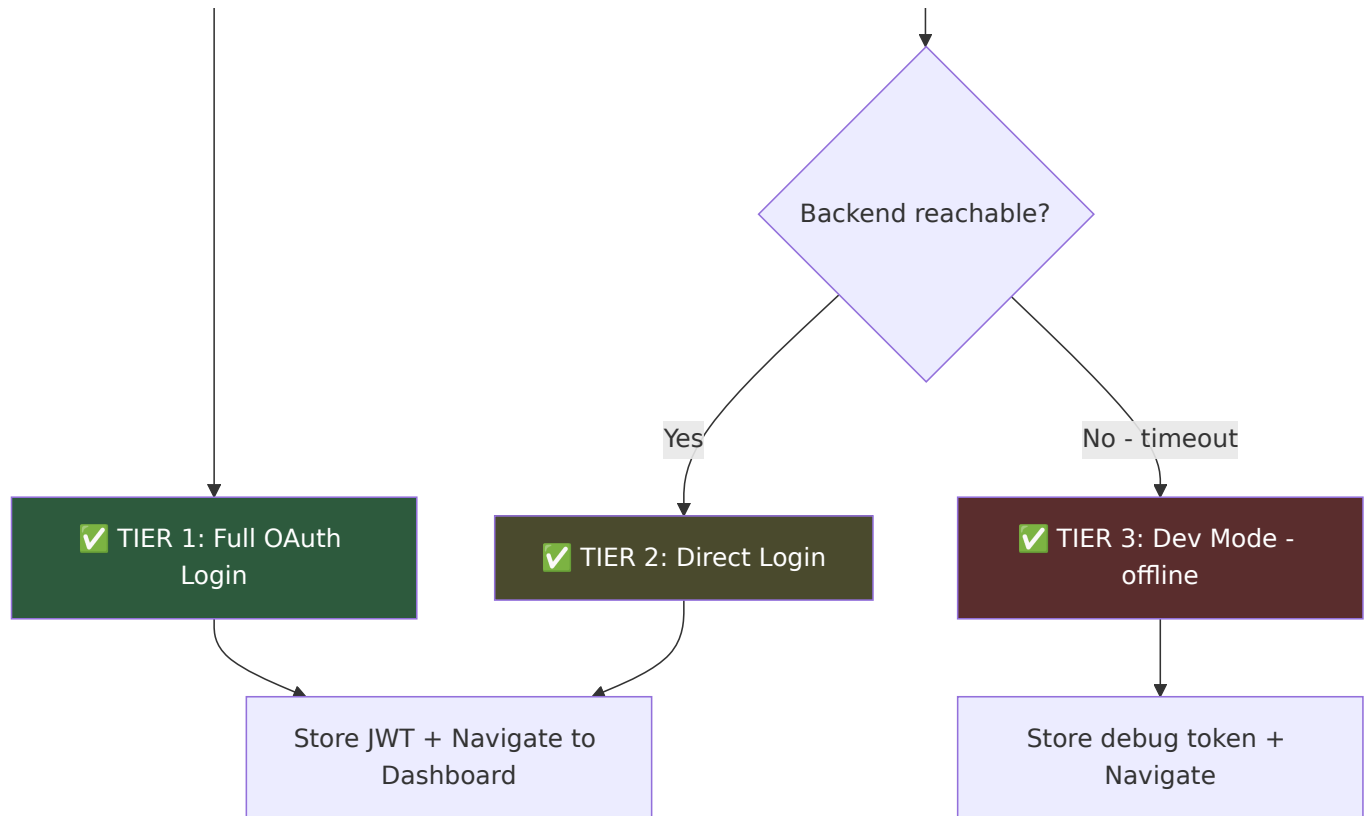
  void setAuthData(int userId, String name, String token) {
    currentUserId = userId;
    userName = name;
    jwtToken = token;
    notifyListeners(); // All widgets rebuild with new auth state
  }
}
```

Token lives in **THREE** places simultaneously:

Location	Persistence	Purpose
FlutterSecureStorage	✅ Survives app restart	Auto-login on next launch
AppProvider.jwtToken	❌ Lost on app close	Quick access during session
HTTP headers (per-request)	❌ Transient	Sent to backend for auth

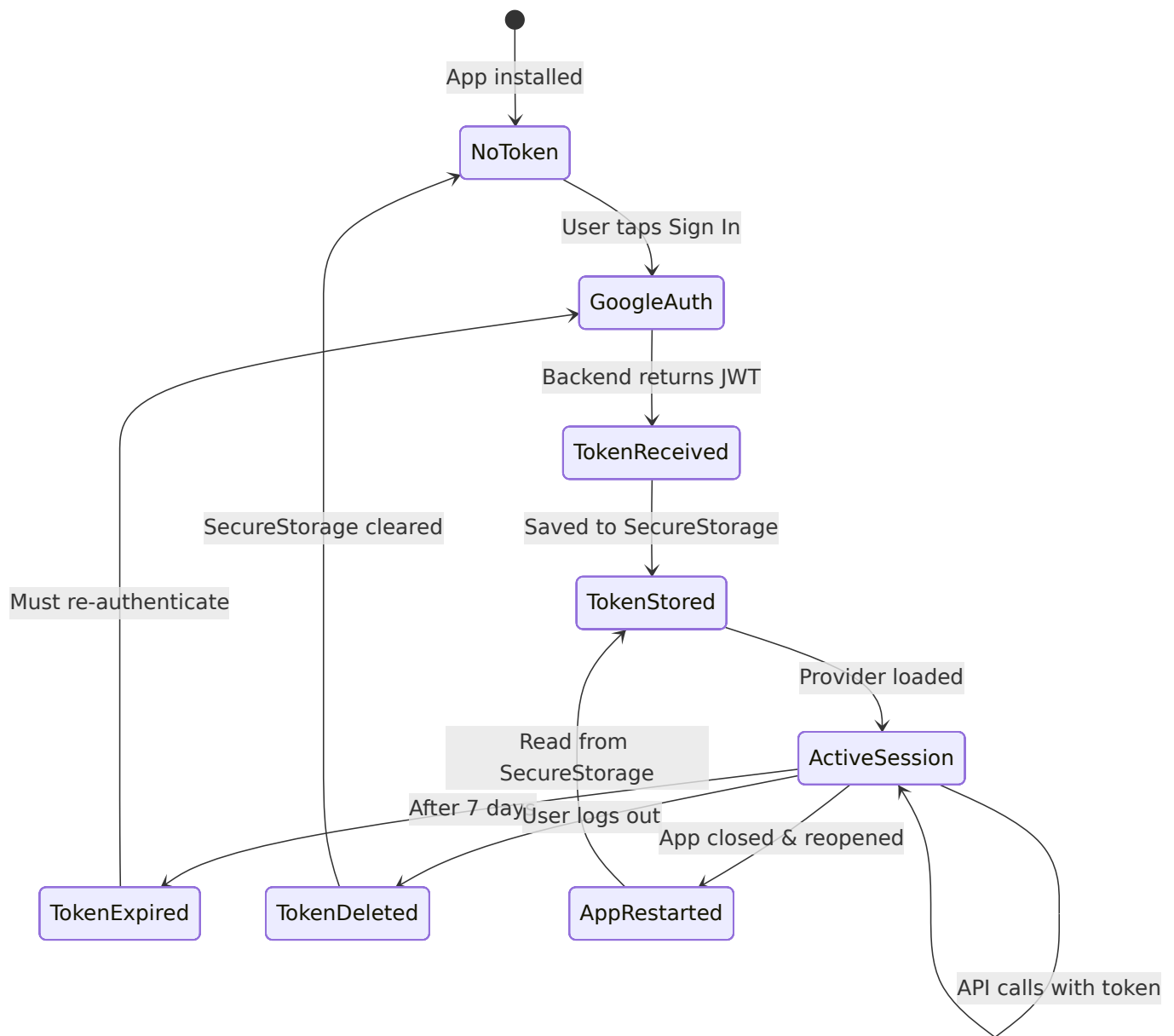
6. The 3-Tier Fallback System





Tier	When It Triggers	Security Level	What Happens
Tier 1	idToken valid + backend up	● Full	Google verifies identity → real JWT from backend
Tier 2	idToken null + backend up	● Good	Google provides email → backend creates user + JWT
Tier 3	Backend unreachable	● Dev only	Hardcoded user_id=1 + debug token

7. Token Lifecycle



Token expiration:

```

# Set at creation (backend)
expiration = datetime.datetime.utcnow() + datetime.timedelta(days=7)

# Token payload includes:
{"user_id": 3, "email": "...", "exp": 1716201600} # Unix timestamp

# When exp < current_time → token is invalid
# User must re-authenticate with Google
  
```

8. Security Considerations

What We Protect Against

Threat	Protection
Token tampering	HMAC-SHA256 signature — any modification invalidates the token
Token theft (on device)	FlutterSecureStorage with AES-256 + Android Keystore
Token theft (in transit)	HTTPS encryption (Render provides SSL)
Replay attacks	Token expiration (7 days)
Forged Google tokens	<code>verify_oauth2_token</code> checks signature + audience + issuer
SQL injection	Parameterized queries (<code>%s</code> placeholders in <code>psycopg2</code>)
Cross-origin attacks	CORS middleware configured

What's Acceptable for a DBMS Demo

Production Would Need	Our Demo Has	Why It's OK
Token refresh mechanism	Single 7-day token	Demo lasts one day
Token blacklist on logout	No server-side invalidation	Stateless by design
Rate limiting	No rate limits	Not public-facing
HTTPS certificate pinning	Standard HTTPS	Demo context

9. Viva-Ready Q&A

Q: Why not just use Google's token directly?

Google's ID token expires in ~1 hour. We'd need to refresh it constantly. Instead, we verify it once, extract the email, and issue our own 7-day JWT. This decouples our app from Google's token lifecycle.

Q: What makes JWT "stateless"?

The server stores no session data. The token itself contains everything needed (`user_id`, `expiration`). Any server with the SECRET key can verify it — no database lookup required.

Q: Can someone decode the JWT and see the payload?

Yes! JWT payload is base64-encoded, not encrypted. But they can't MODIFY it — the HMAC-SHA256 signature would break. We don't put sensitive data (passwords) in the payload.

Q: Why FlutterSecureStorage over SharedPreferences?

SharedPreferences stores data as plain XML. Root users or malware can read it. FlutterSecureStorage encrypts with AES-256, and the encryption key lives in Android's hardware-backed Keystore — inaccessible even with root.

Q: Why HS256 instead of RS256?

HS256 (symmetric) uses one shared key for signing and verifying. Perfect for single-server setups. RS256 (asymmetric) uses public/private key pairs — needed when multiple services verify tokens independently. For our monolithic FastAPI backend, HS256 is simpler and faster.

Q: What happens when the token expires?

The app tries to use it → backend rejects it (PyJWT raises `ExpiredSignatureError`) → app redirects to login screen → user signs in with Google again → new token issued.

Q: How is the user created in the database?

First login: `INSERT INTO Users (name, email) VALUES (?, ?)`. Subsequent logins: `SELECT * FROM Users WHERE email = ?` — finds existing user. The email column has a `UNIQUE` constraint preventing duplicates.